

10주 차 보고서

(2026년 3월 1일 ~ 2026년 3월 7일)

작성자

박신우

이번 주 한 일

3.3~4 캐릭터 모션 매칭 애니메이션 기본 구성

모션 매칭을 적용하기 위해, 우선적으로 포스 서치 스키마를 만들어 주었다.

포스 서치 스키마의 채널 속성에는 2가지 원소가 있다.

이 채널이라는 속성을 이용하여, 애니메이션에 필요한 정보들을 추출해낸다.

Channels	2 Array elements	+	⌵
▶ Index [0]	Trajectory Channel	▼	▼
▶ Index [1]	Pose Channel	▼	▼

Trajectory Channel은 플레이어의 Input값을 통해서
캐릭터의 이동 궤적이 어떻게 될 것인지 정보를 생산해낸다.

▼ Index [0]	Trajectory Channel	▼	▼
Weight	7.0		
▼ Samples	4 Array elements	+	⌵
▼ Index [0]	5 members	▼	
▶ Debug Color			
Offset	-0.4		
Flags	Position XY	▼	
Weight	0.4		
Normalization Group	None		
▼ Index [1]	5 members	▼	
▶ Debug Color			
Offset	0.0		
Flags	Velocity XY Facing Direction XY	▼	
Weight	2.0		
Normalization Group	None		
▼ Index [2]	5 members	▼	
▶ Debug Color			
Offset	0.35		
Flags	Position XY Facing Direction XY	▼	
Weight	0.7		
Normalization Group	None		
▼ Index [3]	5 members	▼	
▶ Debug Color			
Offset	0.7		
Flags	Velocity XY Position XY Facing Direction XY	▼	
Weight	0.5		

2차원 좌표에서 캐릭터의 위치, 2차원 좌표에서 캐릭터의 속도,
앞방향을 바라볼 때 캐릭터의 이동방향 등등 사용해서 캐릭터의 이동 궤적을 예측한다.

이번 주 한 일

Trajectory Channel의 코드도 보고 공부를 한 결과 아래의 구조를 가지고 있었다.

• Samples 설계

- -0.4s : PosXY
- 0s : VelXY + FacingDirXY
- +0.35s : PosXY + FacingDirXY
- +0.7s : VelXY + PosXY + FacingDirXY



(Finalize)

• [SubChannels 생성]

- Position 채널(각 Offset)
- Velocity 채널(각 Offset)
- Heading 채널(각 Offset)
- (모두 XY면 z 제거)



(PoseSearch 비교)

QueryVector vs PoseVector 매칭

Samples 이란

Trajectory 채널이 어느 시점의 어떤 정보 (위치, 속도, 방향)를 feature로 쓸지 적어둔 설계 리스트

설계 리스트가 필요한 이유

모션 매칭은 현재 캐릭터의 상태와 DB에 저장된 애니메이션들을 숫자벡터로 만들어서 비교한다.

그런데 비교를 하려면

- 위치? 방향? 무엇을 비교할지
- 언제의 값을 볼지 (과거, 미래 현재)
- 3D 전체를 할지 XY 평면을 할지

위의 내용을 정해야해서 Samples에 적는다.

Samples 생성자에서

0.4초 위치(XY), 0초 속도+방향(XY), 0.35초 위치+방향(XY), 0.7초 속도+위치+방향(XY)를 비교용 특징으로 쓰겠다 라고 하고 있다.

Finalize() 함수

Finalize 함수는 Samples를 실제 작동하는 특징 채널들로 변환하는 단계이다.

Finalize()가 필요한 이유

모션 매칭 시스템에서 특징을 계산하는 기본 단위는 Position, Velocity, Heading과 같은 개별 채널이다.

Trajectory 채널은 이러한 여러 채널을 하나의 이동 궤적 정보로 묶어 표현한 상위 개념이다. 따라서 Trajectory에서 정의된 샘플 정보를 기반으로 실제 특징 계산에 사용되는 개별 채널들을 생성하는 과정이 필요하다.

Finalize()함수는 Samples에 정의된 Offset과 Flags정보를 바탕으로 Position, Velocity, Heading 채널을 생성하여 SubChannels에 등록하고, PoseSearch 시스템이 사용할 Feature Channel 구성을 최종적으로 완성하는 역할을 수행한다.

SubChannels이란

SubChannels는 Trajectory 채널을 구성하는 실제 특징 계산 채널들의 목록이다.

PoseSearch

PoseSearch의 전체 동작 구조는 코드만으로 어떻게 동작하는지 이해하기 어려웠다.

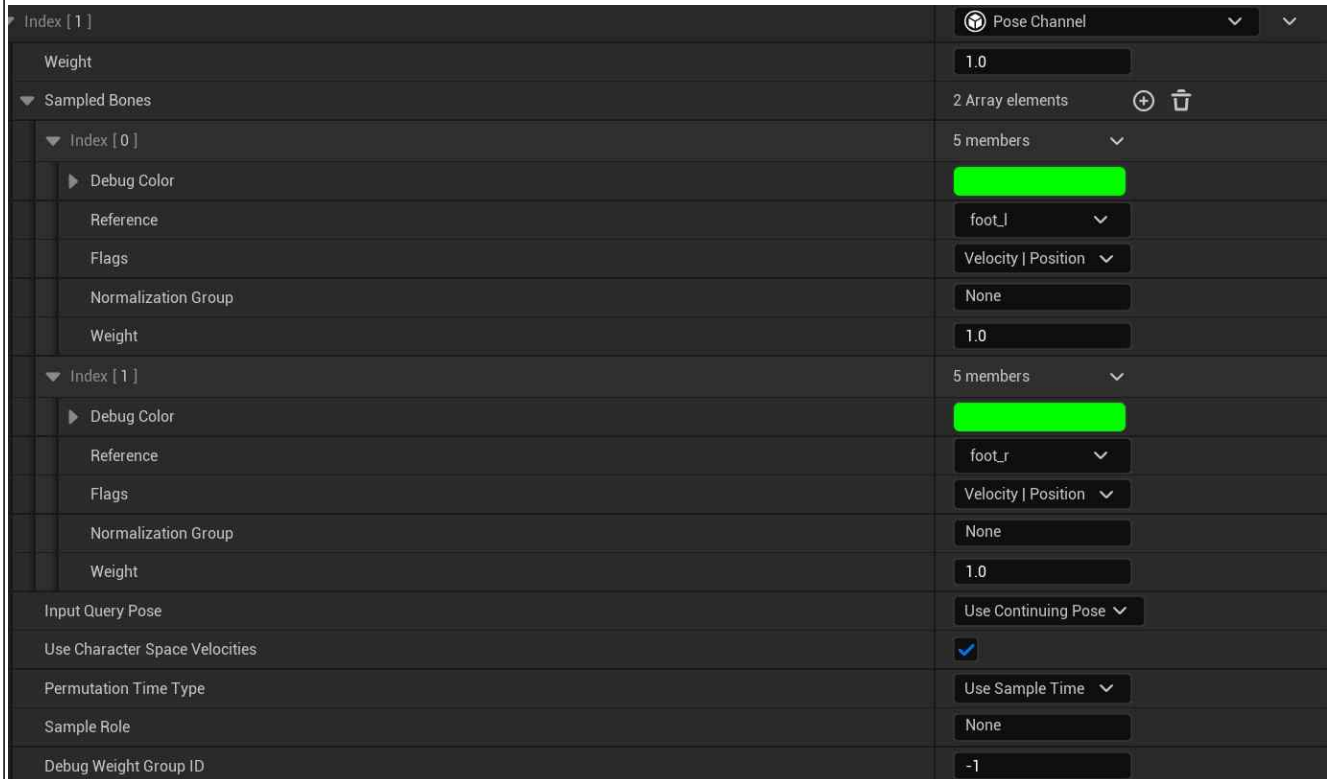
그래서 따로 찾아 보았는데, PoseSearch는 현재 캐릭터 상태로부터 생성된 QueryVector와 데이터베이스에 저장된 PoseVector를 비교하여 가장 유사한 포즈를 선택하는 방식으로 동작한다는 것을 확인하였다.

정리하자면, 캐릭터의 위치, 방향, 속도 등의 정보를 기반으로 이동 궤적을 예측하기 위한 특징 데이터를 생성한다는 것을 알 수 있었다.

이번 주 한 일

Pose Channel은 플레이어 캐릭터가 사용하고 있는 스켈레탈 메시의 본 위치 정보를 추출하는 채널이다.

foot_l 본과 foot_r 본의 위치 정보를 특징으로 사용하여 현재 캐릭터의 발 위치와 가장 유사한 애니메이션 포즈를 선택하는 데 활용된다.



언리얼 엔진에서 기본적으로 Trajectory Channel과 Pose Channel에서 정보를 뽑아내서, 현재 애니메이션과 다음에 예측되는 애니메이션의 유사성을 판단해서 다음 애니메이션을 결정한다고 이해하였다.

그후 실제로 애니메이션을 관리할 Pose Search Database파일을 만들어 주었다.

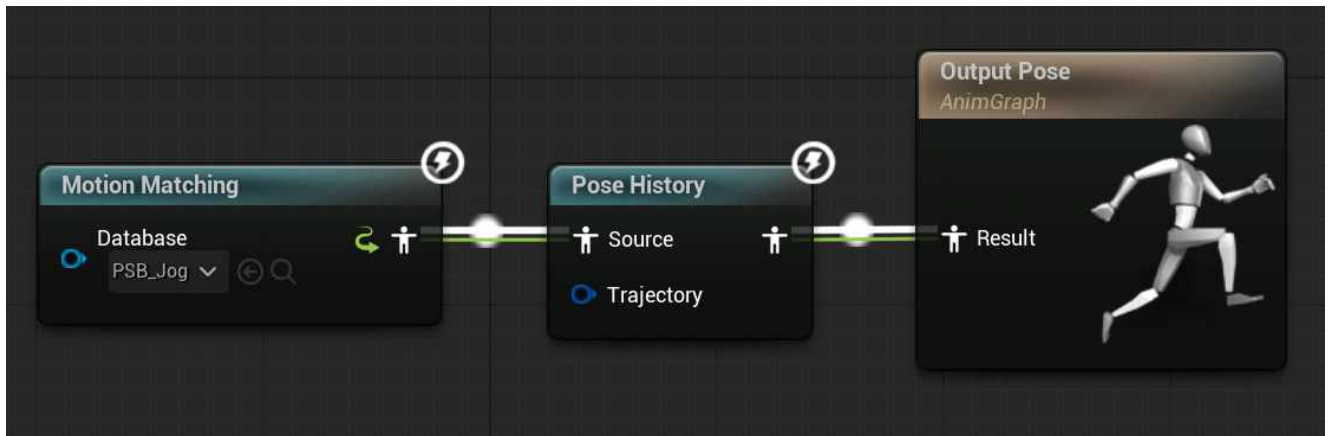
그리고 데이터베이스 파일에 비무장 상태의 Jog 애니메이션 파일들을 추가해주었다.



이번 주 한 일

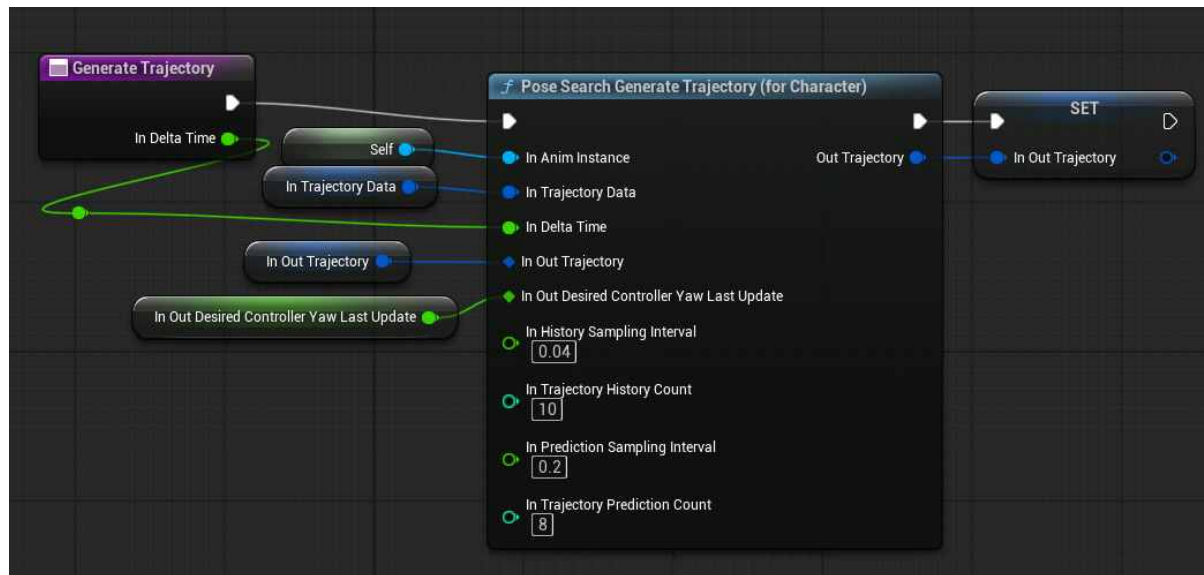
3.5 캐릭터 모션 매칭 애니메이션 적용

모션 매칭을 적용한 애니메이션 블루프린트를 만들어 주었다.



모션 매칭 노드에 Pose Search Database를 넣어주었고
Pose History 노드는 이전 프레임에서 모든 본의 위치를 가지고 있는 노드이다.

Pose History 노드에서 Inpute으로 Trajectory를 받고 있는데,
이를 위해서 함수를 하나 만들어 주었다.



여기서 Pose Search Generate Trajectory 함수는,
모션 매칭이 올바른 애니메이션을 선택할 수 있도록 캐릭터의 이동 Trajectory 데이터를
매 프레임마다 생성 및 갱신하는 역할을 해준다.

이런식으로 비무장 상태에서의 걷기 역할을 하는 애니메이션 블루프린트를 만들었다.

적용을 해보니 잘 적용이 되었지만 아직은 캐릭터의 실제 이동 속도는 0에서 점진적으로
가속되는 반면, 재생되는 애니메이션은 이미 전속력으로 달리는 루프 포즈이기 때문에
발이 땅을 미끄러지듯 보이는 현상이 발생하였다.

이번 주 한 일

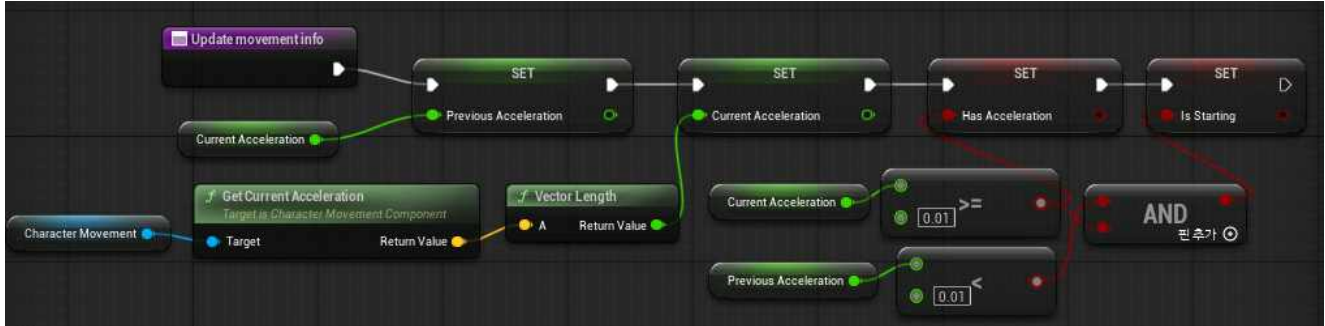
3.6 Start 애니메이션 추가

위에 설명한 문제들을 해결하기 위하여,

정지 → 달리기 출발 구간을 담당하는 Start 애니메이션을 추가하기로 하였다.

Pose Search Database 파일을 만들어 Start 애니메이션들을 넣어 주었다.

애니메이션 블루프린트에서 움직임 상태를 관리해주는 함수를 만들어주었다.



그후 Chooser Table을 만들어 주었다.

선택기 테이블		CT_Unarmed	
결과	부울	부울	부울
	Has Acceleration	Is Starting	
PSD_Jog	True	False	
PSD_JogStart	True	True	
PSD_Idle	False	False	
PSD_Idle			

속도가 없고 애니메이션 시작하지 않는다면 Idle,

속도가 있고 애니메이션 시작한다면 JogStart,

속도가 있고 애니메이션 시작중이 아니라면 Jog가 되도록 하였다.

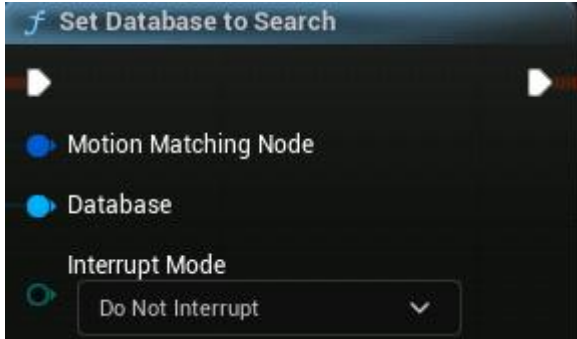
이제 이 Chooser 매 업데이트마다 가져오는 함수를 애니메이션 블루프린트에 만들어줬다.



이번 주 한 일

근데 키보드에서 손을 떼도 Idle 애니메이션이 아닌 걸고 있는 애니메이션이 출력이 되는 문제가 발생하였다.

원인을 찾아보니, Interrupt Mode를 Do Not Interrupt로 해놓아서 데이터베이스간에 전환이 안 된 것이었다.



3.6 보고서 정리

다음 주 할 일

- 모션 매칭 문제 해결
- 비무장 상태와 무장 상태 애니메이션 연결하기

특이사항

10주 차 보고서

(2026년 3 월 01 일 ~ 2026년 3 월 07 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일

```
pushd %~dp0
protoc.exe -I=./ --cpp_out=./ ./Enum.proto
protoc.exe -I=./ --cpp_out=./ ./Struct.proto
protoc.exe -I=./ --cpp_out=./ ./Protocol.proto

GenPackets.exe --path=./Protocol.proto --output=ClientPacketHandler --recv=C_ --send=S_
GenPackets.exe --path=./Protocol.proto --output=ServerPacketHandler --recv=S_ --send=C_

IF ERRORLEVEL 1 PAUSE

XCOPY /Y Enum.pb.h "../../../GameServer"
XCOPY /Y Enum.pb.cc "../../../GameServer"
XCOPY /Y Struct.pb.h "../../../GameServer"
XCOPY /Y Struct.pb.cc "../../../GameServer"
XCOPY /Y Protocol.pb.h "../../../GameServer"
XCOPY /Y Protocol.pb.cc "../../../GameServer"
XCOPY /Y ClientPacketHandler.h "../../../GameServer"

XCOPY /Y Enum.pb.h "../../../DummyClient"
XCOPY /Y Enum.pb.cc "../../../DummyClient"
XCOPY /Y Struct.pb.h "../../../DummyClient"
XCOPY /Y Struct.pb.cc "../../../DummyClient"
XCOPY /Y Protocol.pb.h "../../../DummyClient"
XCOPY /Y Protocol.pb.cc "../../../DummyClient"
XCOPY /Y ServerPacketHandler.h "../../../DummyClient"

DEL /Q /F *.pb.h
DEL /Q /F *.pb.cc
DEL /Q /F *.h

PAUSE
```

```

import argparse
import Jinja2
import ProtoParser

def main():

    arg_parser = argparse.ArgumentParser(description = 'PacketGenerator')
    arg_parser.add_argument('--path', type=str, default='C:/Users/0412j/Server Part4/Server/Common/Protobuf/bin/Protocol.proto', help='proto path')
    arg_parser.add_argument('--output', type=str, default='TestPacketHandler', help='output file')
    arg_parser.add_argument('--recv', type=str, default='C_', help='recv convention')
    arg_parser.add_argument('--send', type=str, default='S_', help='send convention')
    args = arg_parser.parse_args()

    parser = ProtoParser.ProtoParser(1000, args.recv, args.send)
    parser.parse_proto(args.path)
    file_loader = Jinja2.FileSystemLoader('Templates')
    env = Jinja2.Environment(loader=file_loader)

    template = env.get_template('PacketHandler.h')
    output = template.render(parser=parser, output=args.output)

    f = open(args.output+'.h', 'w+')
    f.write(output)
    f.close()

    print(output)
    return

if __name__ == '__main__':
    main()

```

파일을 생성해주는 배치파일과, 패킷을 자동적으로 만들어주는 파이썬파일을 만들었다. 배치파일을 실행시키면 구글 protobuf에 필요한 Enum.pb.h / Enum.pb.h / Enum.pb.cc 등등을 기존에 있던 것을 삭제하고 새롭게 필요한 부분으로 생성해준다.

```

syntax = "proto3";
package Protocol;

enum PlayerType
{
    PLAYER_TYPE_NONE = 0;
    PLAYER_TYPE_KNIGHT = 1;
    PLAYER_TYPE_MAGE = 2;
    PLAYER_TYPE_ARCHER = 3;
}

```

```

syntax = "proto3";
package Protocol;

import "Enum.proto";
import "Struct.proto";

message C_LOGIN
{
}

message S_LOGIN
{
    bool success = 1;
    repeated Player players = 2; // 아이디 발급 전
}

message C_ENTER_GAME
{
    uint64 playerId = 1;
}

message S_ENTER_GAME
{
    bool success = 1;
}

message C_CHAT
{
    string msg = 1;
}

message S_CHAT
{
    uint64 playerId = 1;
    string msg = 2;
}

```

```

syntax = "proto3";
package Protocol;

import "Enum.proto";

message Player
{
    uint64 id = 1;
    string name = 2;
    PlayerType playerType = 3;
    // 스탯 정보...
}

```

```

class ProtoParser():
    def __init__(self, start_id, recv_prefix, send_prefix):
        self.recv_pkt = [] # 수신 패킷 목록
        self.send_pkt = [] # 송신 패킷 목록
        self.total_pkt = [] # 모든 패킷 목록
        self.start_id = start_id
        self.id = start_id
        self.recv_prefix = recv_prefix
        self.send_prefix = send_prefix

    def parse_proto(self, path):
        f = open(path, 'r')
        lines = f.readlines()

        for line in lines:
            if line.startswith('message') == False:
                continue

            pkt_name = line.split()[1].upper()
            if pkt_name.startswith(self.recv_prefix):
                self.recv_pkt.append(Packet(pkt_name, self.id))
            elif pkt_name.startswith(self.send_prefix):
                self.send_pkt.append(Packet(pkt_name, self.id))
            else:
                continue

            self.total_pkt.append(Packet(pkt_name, self.id))
            self.id += 1

        f.close()

class Packet:
    def __init__(self, name, id):
        self.name = name
        self.id = id

```

프로토 파일들을 읽을 수 있는 데이터 파서 역할을 할 수 있게 만들어주었다. 서버와 클라의 프로토콜을 자동화하기 위한 핵심 역할이다.

```
bool Handle_S_CHAT(PacketSessionRef& session, Protocol::S_CHAT& pkt)
{
    std::cout << pkt.msg() << endl;
    return true;
}
```

기존에 있던 서버 패킷 핸들러와 클라이언트 패킷 핸들러를 이용하여 간단한 채팅 시스템을 만들었다.

다음 주 할 일

여태까지 만든 서버 프레임워크로 클라이언트에 붙여볼 예정이다. 구글 protobuf를 이용하기 때문에 클라이언트에서도 구글 protobuf를 인식할 수 있게 수정을 할 것이다.

10주 차 보고서

(2026년 3 월 1 일 ~ 2026년 3 월 7 일)

작성자

안윤진

이번 주 한 일

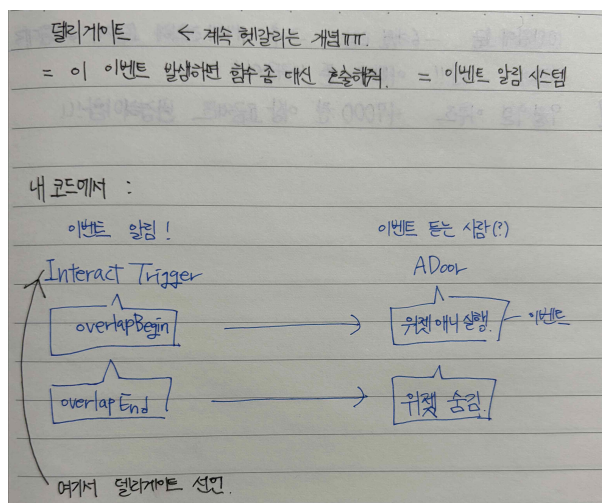
- 코드에 문제가 없는데 빌드로 언리얼 에디터가 안열리고 오류가 난다면, 다른 파일일지라도 언리얼 자체를 다 달고 다시 실행해야 한다. 요즘 이런 오류로 시간 지체가 많다.



: 신우형이 제작해둔 상호작용 트리거를 사용해 문 액터를 만들었다. 이때 문을 일반버전과 위로 열리는 버전 2개를 만들 것이라, 열리는 방향을 블루프린트에서 변수로 세팅할 수 있게 해주었다.

컨테이너 주변에 버튼까지 함께 추가한 문 액터 두 종류를 만들었고, 각 집 블루프린트에 배치하였다.

위에 같은 컨테이너 형식의 집은 버튼 주변에 가 상호작용키를 눌러야 문이 열리고 닫힌다.



```
//델리게이트 선언 (상호작용 ui 띄우기 위해, 오버랩 시작, 끝에 이벤트 발생)
DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(FOnInteractEnter, AActor*, OtherActor);
DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(FOnInteractExit, AActor*, OtherActor);

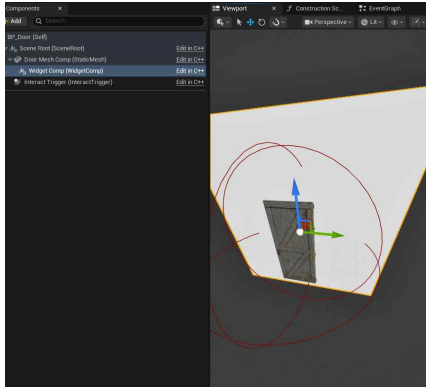
UCLASS()
class FPSPROJECT_API UInteractTriggerComponent : public USphereComponent
```

```
void AADoor::BeginPlay()
{
    Super::BeginPlay();
    if (WidgetComp) WidgetComp->InitWidget();

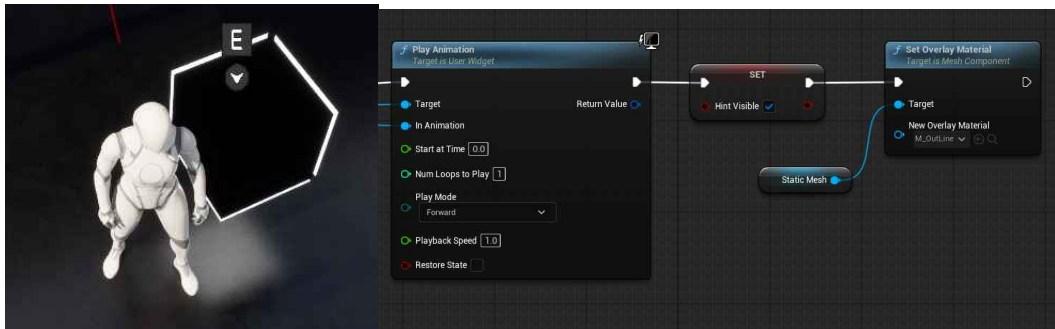
    OriginalRotation = DoorMeshComp->GetRelativeRotation(); //문이 처음에 어떤 방향을 향하고 있는지 저장
    Target = OriginalRotation; // 처음 목표도 원래 각도
    //델리게이트 바인딩 (트리거 컴포넌트의 OnEnter 이벤트를 들도록 설정)
    InteractTrigger->OnEnter.AddDynamic(this, &AADoor::WidgetStart);
    InteractTrigger->OnExit.AddDynamic(this, &AADoor::WidgetEnd);
}
```

트리거 범위에 들어오자마자 위젯 이미지를 띄우기 위해 델리게이트로 오버랩 시작과 끝에 door에 있는 위젯 함수를 바인딩 해주었다. 또 위젯에도 시작과 끝(역방향 재생) 애니메이션을 설정해 좀 더 게임 상호작용 같은 효과를 주었다.

이번 주 한 일



- 문 액터에 상호작용 위젯 표시에서 문제 발생, 위젯이 제대로 그려지지 않고 양면도 아닌 단편으로 흰 박스가 그려지는 문제.



- 기존에 만들어둔 머터리얼이 액터 아웃라인 표시가 제대로 되지 않음. (기존 구현한 블루프린트)

: 원래 프로젝트 들어가서 확인하니 오버레이 머터리얼을 새로 추가해 그린 것이지, 위젯 머터리얼에 추가한 것이 아니었음. 문 액터 class에 변수로 오버레이 머터리얼 추가, 그 후 머터리얼을 적용하니 제대로 구현됨.



문이 열리고 닫히는 과정에도 제대로 아웃라인을 표시함. 오버랩 시작과 끝 함수에 오버레이 머터리얼 셋팅 해 범위 밖으로 나가면 보이지 않음

: 위젯 표시 문제의 저 박스도 잘못된 경로에 머터리얼을 추가해서 생긴 문제. 머터리얼을 기본으로 돌려주니 상호작용 표시가 잘 나타남. 저 흰박스는 위젯이 그려지는 크기였음. 위젯 블루프린트에서 위젯의 이미지 크기가 너무 커서 그것보다 작은 드로우 사이즈에 꽂아서 그려지게 된 것. 위젯 자체의 크기를 줄여주니 모든 이미지가 픽워짐.

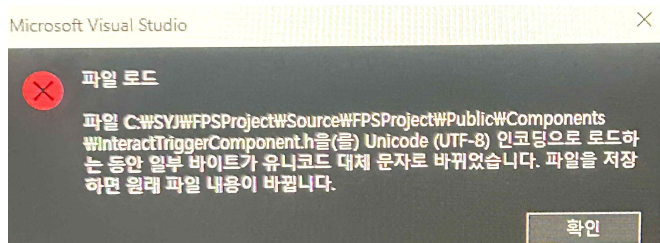
- 상호작용 UI가 같이 이동하지 않음. 그려지긴하나 뒷면이 보이는 등 스크린이 아닌 월드에 고정된 느낌으로 구현됨.

: `WidgetComp->SetWidgetSpace(EWidgetSpace::Screen);` 스크린으로 변경하니 뒷면이 보이지 않게 됨.

이번 주 한 일

- `SetText(FText::FromString(TEXT("문 열기")));` 이 함수만 사용하면 오류가 생겼는데, ai나 인터넷을 뒤져도 원인을 모르겠음. 다 이렇게 사용함.

: 정말 오랜시간 걸려 깨달았다. 한글을 사용해서 안되는 것이었다... 영어나 숫자로 변경하니 바로 잘 됨.



한글 사용 가능 방법을 찾아보니 윈도우 11에선 beta:Unicode UTF-8사용으로 설정을 바꿔주기만해도 사용 가능하да해 그렇게 변경하고 실행시켰더니 한글 사용이 가능했고 제대로 띄워졌다. 하지만 파일에 적혀있던 모든 한글들이 전부 변환되어 읽을 수 없게 되었다. 그래서 신우형과 상의 후 다시 원래대로 돌리고 영어를 쓰던 다른 방법을 사용하기로 하였다.

최종 문은 이렇게 제작되었다.



- 상호작용 ui 이미지 변경

상호작용키가 F로 정해지면서, 원래 E키로 만들었던 이미지를 넣고 ai에게 F키로 이미지 변경을 요청했다. 어렵지 않은 작업이라 그런지 한 두 번 구박하니 잘 나왔다.

다음 주 할 일
이제 정말 맵 끝내기..

특이사항